



北京工商大学
BEIJING TECHNOLOGY AND BUSINESS UNIVERSITY

源代码级异步操作 系统调试方法

面向 Rust 操作系统的跨特权级与异步统一调试

团队成员：曾小红 · 王浩铭 · 武雪妍 指导教师：吴竞邦

北京工商大学 · 计算机与人工智能学院 · 2026 年 8 月

项目目录

- 01 **项目背景**
赛题与调试器背景、项目增量
- 02 **总体目标**
工作目标与项目总览
- 03 **关键技术实现**
五个技术方案分述
- 04 **测试验证**
四平台验证与完成情况

项目背景



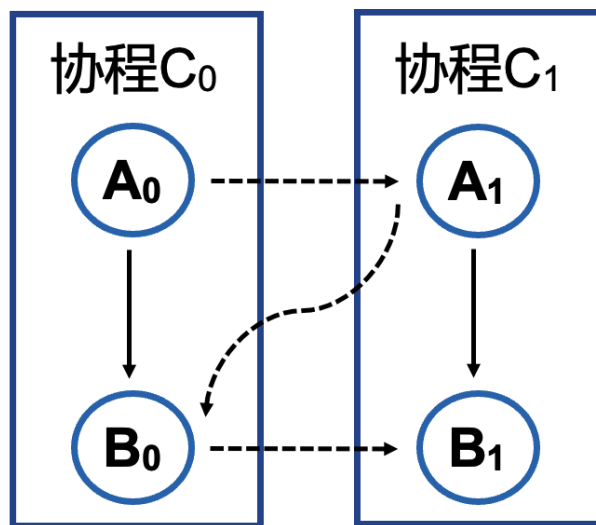
赛题

proj55-源代码级内核调试器



项目背景

- 操作系统调试涉及内核态与用户态之间的**特权级切换**，调试时**需手动切换符号表与断点组**。
- 在**Rust异步程序**的调试场景中，调试器的调用栈中**多协程执行流混合**，无法识别各异步协程的**独立调用链与等待关系**。



实际执行流: $A_0 \rightarrow A_1 \rightarrow B_0 \rightarrow B_1$

传统GDB显示的调用栈: A_0B_0 或 A_1B_1

逻辑执行流:
$$\left\{ \begin{array}{l} C_0 : A_0 \rightarrow B_0 \\ C_1 : A_1 \rightarrow B_1 \end{array} \right.$$

项目背景：项目增量



历届完成阶段

- 2023年：针对**Rust语言同步函数**调试以及**跨特权级断点**调试
- 2024年：针对C语言进行扩充以及实际开发板**硬件调试**的支持
- 2025年：针对**Rust语言异步函数的跟踪调试**进行设计和开发



今年增量

- ✓ 优化异步跟踪方案，将异步跟踪从**静态预分析**改为运行时**按需发现**；
- ✓ 实现**异步跟踪**与**特权级切换**的融合；
- ✓ 完成 VS Code **图形化插件**；
- ✓ 面向不同操作系统的通用化改进，将调试器从教学 OS 推广至**组件化 OS 和异步 OS**。

解决的关键问题

1. 取消调试前静态分析调试信息模块，从根本上解决原异步跟踪方案插桩开销大的问题

- ✓ 运行时按需发现并插桩、区分同一类型的不同实例、引入trace root概念

2. 设计 save/restore 机制，解决异步跟踪与跨特权级调试冲突

- ✓ 实现异步操作系统调试功能、基于 async-os 验证

3. 针对调试功能设计并实现展示界面，将跟踪结果可视化

- ✓ vs code 插件化、面板实现

4. 通过三套技术优化方案，增强调试器的移植适配能力

- ✓ 通用性改进、支持调试组件化OS、基于 StarryOS 验证

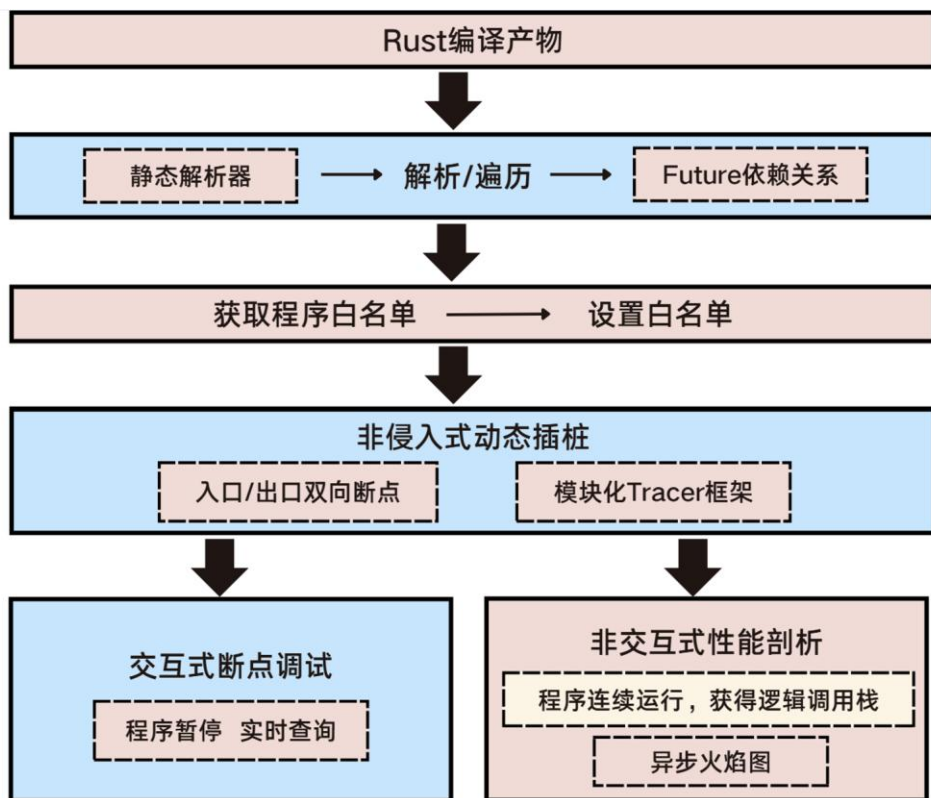
目标拆解

针对需要解决的四个核心问题进行目标拆解

核心问题	最终目标	技术项
全量插桩开销大	取消调试前静态分析，改为运行时按需发现	运行时按需发现机制
		Trace root 与白名单动态管理机制
异步跟踪与跨特权级调试冲突	异步跟踪与跨特权级调试融合，实现异步操作系统调试	OS 调试能力移植
		Save/Restore 跨特权级异步跟踪数据保持
异步跟踪结果不直观	图形化界面设计，跟踪结果可视化	实现 VS Code 插件化架构
		Async Inspector 图形化面板
跨特权级调试通用性差	针对组件化OS调试的通用化改进	函数名断点 + 通用收敛点 + 方向属性
		动态进程组自动注入
		跨 Rust 版本变量读取

技术实现一：异步跟踪方案改进

问题分析



原异步跟踪方案：

- 调试前**需静态分析调试信息**以获取poll函数间的依赖关系
- 会对白名单中所有的函数及其相关函数进行**全量插桩**
- 仅依赖DWARF调试信息**无法区分同一类型的不同实例**

流程复杂、插桩开销大、实例混淆

技术实现一：异步跟踪方案改进

具体优化方案

流程复杂



- 取消调试前的静态分析，改为运行时按需从poll函数__awaitee字段获取依赖函数

插桩开销
大



- 引入 Trace root 概念，即「从哪个函数开始追踪」
- 取消了对白名单中函数及其相关函数的全量插桩
- 追踪范围被精确限定在从 trace root 出发、沿 await edge 和 call edge 实际执行到的路径上

实例混淆



- 原方法使用DWARF 中 Future 类型的偏移量来生成标识符——这只能区分不同的 async 函数类型，无法区分同一类型的不同实例
- 改进使用 (poll_symbol, env_ptr)二元组作为协程实例的唯一标识

技术实现二：异步跟踪与跨特权级调试融合

问题分析

核心冲突：特权级切换会清除异步跟踪数据，本质是两个功能的相关数据生命周期存在冲突

■ 存在冲突的相关数据有：

1. **断点数据**：异步跟踪和跨特权级调试共涉及6大类断点
2. **协程状态数据**：包括实例映射表、协程ID、poll次数等
3. **追踪记录数据**：trace root 函数集合、已安装特殊断点函数的集合等
4. **白名单数据**：白名单中函数的地址映射

■ 解决方案：

设计 save/restore 机制，正确维护所有数据的生命周期

技术实现二：异步跟踪与跨特权级调试融合

具体方案

■ save 机制：

执行时间：在清除旧断点组之前执行

- ✓ 保存 poll 入口断点元数据
- ✓ 保存协程状态数据
- ✓ 保存追踪函数列表

■ restore 机制：

执行时间：在新符号表加载完成、新断点组激活后执行

- | | |
|--------------|---------------------------------|
| ① 恢复白名单配置 | 标记地址映射待重建 |
| ② 恢复追踪根与追踪函数 | ACTIVE_ROOTS、CallSite 安装记录 |
| ③ 恢复协程状态 | ID 映射、元数据、poll 序列、TLS 栈 |
| ④ 重建入口断点 | 遍历 poll_entries, 重建 PollEntryBP |

关键点：恢复机制必须按 ① → ④ 顺序执行

顺序的设计依据是：

- A. 白名单和地址映射是基础——没有它，后续任何断点都无法正确定位到目标函数；
- B. 追踪函数列表决定了哪些 poll 入口需要恢复——先知道要恢复谁，再恢复数据；
- C. 协程状态是纯内存数据，不涉及 GDB 断点操作，可以在断点恢复前完成；
- D. PollEntryBP（入口断点）的实际设置必须在最后——它依赖前三个步骤的全部结果；

技术实现三：图形化界面设计与实现

问题分析

```
[rust-future-tracing] Loaded runtime plugin: tokio (currently does nothing)
(gdb) find-poll-fn
[rust-future-tracing] Finding all poll methods...
[rust-future-tracing] Using regex: ^\s*(\d+):\s+(.*)? -> core::task::poll::Poll<(.*)>;$
[rust-future-tracing] Found 8 poll methods.
[rust-future-tracing] Poll map written to: results/poll_map.json
[rust-future-tracing] Please edit this file to select the futures you want to trace.
(gdb) init-dwarf-analysis ../embassy/examples/std/target/debug/tick
Loading DWARF information from: ../embassy/examples/std/target/debug/tick
Successfully initialized DWARF analysis for: ../embassy/examples/std/target/debug/tick
Found 210 compilation units
You can now access the tree with: python tree = gdb.dwarf_tree
Or access DWARF info with: python info = gdb.dwarf_info
(gdb) start-async-debug
[rust-future-tracing] Step 1: Reading user-selected interesting functions...
[rust-future-tracing] Found interesting poll function: static fn tick:____embassy_main_task::{async_fn#0}(*mut core::ta
::Context)
[rust-future-tracing] Found interesting poll function: static fn tick:___led_task_task::{async_fn#0}(*mut core::task::wa
ext)
[rust-future-tracing] Found interesting poll function: static fn tick:___sensor_task_task::{async_fn#0}(*mut core::task:
ontext)
[rust-future-tracing] Mapped static fn tick:____embassy_main_task::{async_fn#0}(*mut core::task::wake::Context) -> tick
bassy_main_task::{async_fn_env#0} (DIE offset: 62764)
[rust-future-tracing] Mapped static fn tick:___led_task_task::{async_fn#0}(*mut core::task::wake::Context) -> tick:___le
ask::{async_fn_env#0} (DIE offset: 61923)
```

Asynchronous Backtraces

```
====
Process 16016:
Thread 16016:
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick:____embassy_main_task::{async_fn_env#0}>>' (ID: 58884):
    Stack is empty.
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick:___led_task_task::{async_fn_env#0}>>' (ID: 58854):
    Stack is empty.
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick:___sensor_task_task::{async_fn_env#0}>>' (ID: 58824):
    tick:___sensor_task_task::{async_fn_env#0}
    -> tick::read_and_publish::{async_fn_env#0}
    -> tick::publish_data::{async_fn_env#0}
=====
```

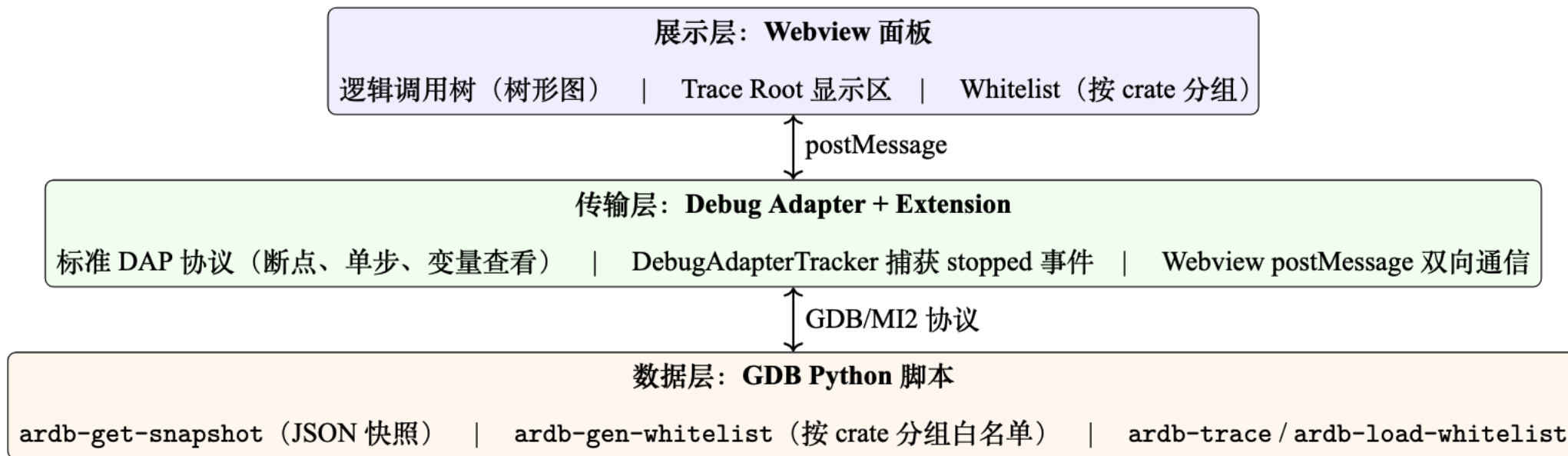
- 25年实现的异步跟踪工具是**命令行版**；
- 开发者在终端中手动输入命令、查看文本输
出来理解异步执行流，**使用门槛高**；
- 此外 VS code **原生调试面板对异步跟踪语义支持较差**；



- ✓ 将跟踪工具**插件化**，降低使用门槛
- ✓ 针对 Rust 异步语义**设计可视化面板**
- ✓ 支持快速**查看异步函数元数据**

技术实现三：图形化界面设计与实现

插件架构图



核心工作:

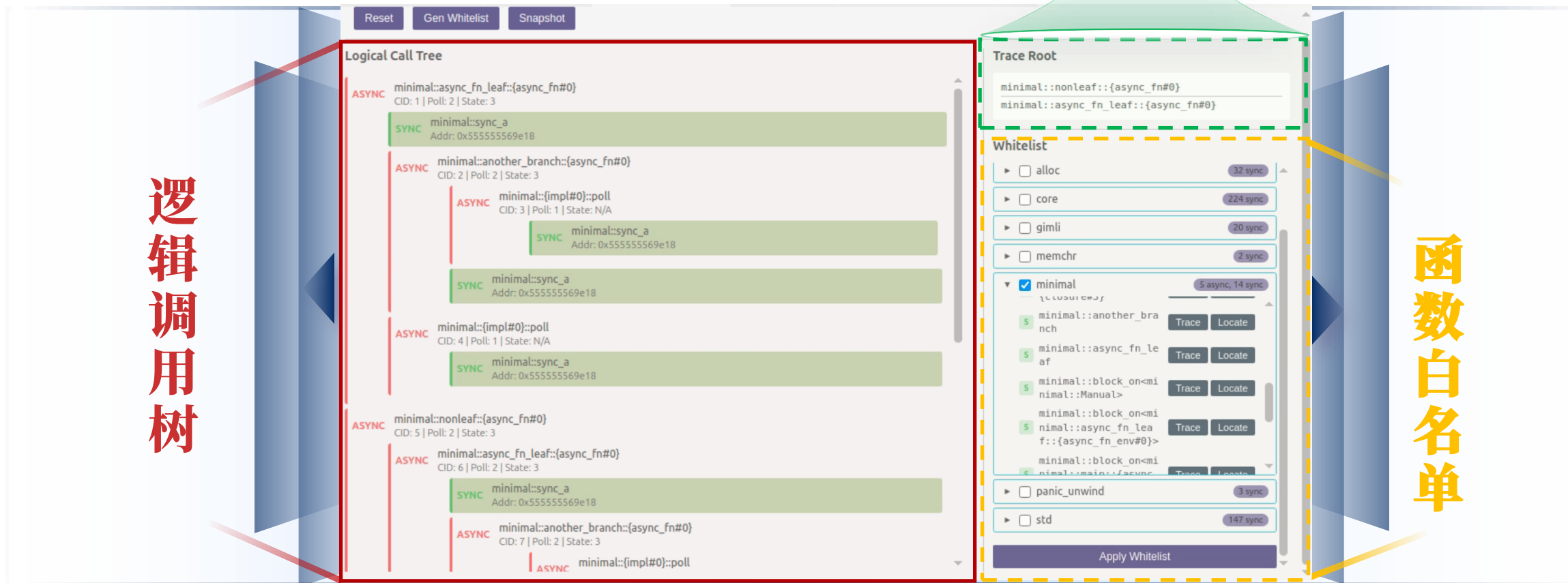
1. 使用 vs code 提供的标准 DAP 协议重构跟踪工具的传输层
2. 插件面板设计与实现

技术实现三：图形化界面设计与实现

追踪根函数

逻辑调用树

函数白名单



- ✓ **Logical Call Tree:** 以**树形图**展示逻辑调用关系；红色节点为 async 协程，绿色节点为 sync 函数；异步节点标注协程 ID、poll 次数、运行状态；点击节点跳转对应源码行。
- ✓ **Trace Root 显示区:** 展示当前已设置的**追踪根函数列表**。
- ✓ **Whitelist:** 按 crate 分组展示可追踪函数，每个函数提供 **Trace** 和 **Locate** 按钮。

技术实现三：图形化界面设计与实现

界面工作流程（蓝色为用户操作，绿色为系统响应）

开发者从启动调试到查看异步执行流：

- 六步操作均在 VS Code 界面内完成；
- 开发者无需切换终端、无需手动编辑配置文件

异步追踪从配置到可视化形成完整闭环



技术实现四：面向不同操作系统的通用化改进

问题分析

- 教学 OS 的静态特征（源码同工作区、单一系统调用入口、进程提前已知）构成了调试方案的隐含前提
- 在 Rust 操作系统领域，**组件化 OS** 是一类重要且普遍的目标，**其灵活架构使得这些前提全部失效**
- 因此需要**拓宽调试方案的通用性**，使其覆盖更多类型的操作系统

问题	说明	在组件化 OS 中的限制
特殊断点依赖本地源码	调试器通过程序路径 + 行数配置特殊断点，这要求本地具有对应源代码	部分特殊断点位于外部 crate，不在本地源码工作区，断点无法设置
硬编码解析进程名	硬编码解析导致对不同 Rust 版本适配度极低	字段布局随 Rust 版本变化，硬编码解析失效
进程组切换依赖提前配置	要求调试开始前将所有用户进程组写入配置文件	运行时启动的程序不可预知，无法提前写入配置

技术实现四：面向不同操作系统的通用化改进

改进方案

➤ 改进一：用函数名指定特殊断点，摆脱对本地源码文件路径的依赖

GDB 本身具备通过符号表查找函数地址的能力，实现用函数名指定特殊断点，只要目标函数被链接到了 ELF 中就能定位

➤ 改进二：放弃硬编码解析字段路径，改为依赖 GDB 提供的稳定接口读取变量

具体方案分三步：

1. 通过 GDB 的 `p < 变量名 >` 获取格式化的变量输出
2. 在传输协议层自建控制台输出捕获机制：
3. GDB 虽然未提供直接获取命令结果的接口，且无法直接判断控制台某行输出属于哪条命令；
4. 但是 GDB 的每一条命令结果输出后都会有以 `^done` 或 `^error` 开头的响应，标志着该命令的处理结束；
5. 我们基于这一观察，实现了控制台输出捕获机制
6. 从输出中提取地址和长度，直接读取目标内存

➤ 改进三：进程组管理从静态预配置到运行时动态自动注入

针对动态创建的进程组实现各类断点的自动注入

测试验证：状态机单元测试与集成测试

通过自动化测试验证调试器状态机核心逻辑和完整调试流程的正确性

通过**37个OSStateMachine 单元测试**与**4个MockMI2 集成测试**快速检验调试器跨特权级调试能力

➤ **单元测试覆盖 四个状态机 的所有合法状态转换路径和边界条件：**

- 1. 正常转换路径：kernel → kernel_single_step_to_user → user → user_single_step_to_kernel → kernel；
- 2. PC 快速路径：当状态机在 user 态、PC 已位于内核空间且栈帧函数名匹配 user→kernel边界断点时，直接触发状态切换，跳过逐指令单步；
- 3. 异常路径：断点组未就绪、符号文件加载失败、GDB 断点编号异步返回超时等异常场景下的状态机行为；

➤ **通过 MockMI2 模拟 GDB 后端，提供可控的寄存器值、栈帧信息和断点编号，在无需真实 QEMU 环境的情况下验证四个集成测试场景：**

- 1. 内核启动 → 首次进入用户态；
- 2. Hook 断点动态进程发现；
- 3. 用户态 → 内核态的 PC 快速路径；
- 4. 多次特权级切换的完整性

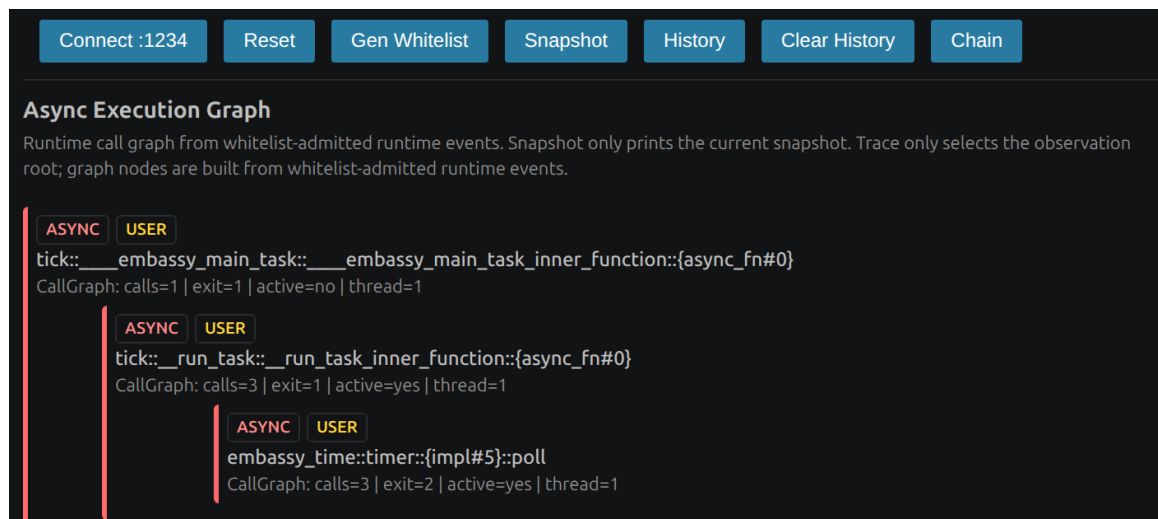
测试验证：embassy 嵌入式异步框架

验证异步跟踪运行时无关性

embassy异步框架：使用自定义异步调度器，不依赖 Tokio等运行时。**通过调试器在 embassy 上正常工作验证异步跟踪不绑定特定异步运行时**

验证结果：

- 三层嵌套等待链 | `main_task → run_task → timer::poll` | `__awaitee` 字段按需发现了全部依赖关系；
- 不同 poll 次数 | `calls=1 / 3 / 3` | 外层单次 poll，内层多次 poll，符合计时器驱动的执行特征；
- 实例级状态区分 | `active=no / yes / yes` | 内层任务已完成，外层仍在等待；



调试演示视频：

[点击查看：embassy 调试演示](#)

测试验证：组件化OS StarryOS

验证组件化 OS 跨特权级调试能力

StarryOS：基于 ArceOS 框架的组件化 Rust OS，其源码组织、系统调用路径、进程模型与rCore 存在根本差异。

验证过程：内核启动后 kernel→user 切换、Hook 断点在 execve 处命中创建进程组、调试器状态机切换、多次切换符号文件正确卸载加载。

1. 内核态符号表自动加载成功：

```
add symbol table from file "/home/iristseng/data/StarryOS/target/riscv64gc-unknown-none-elf/release/starryos" at
.text_addr = 0xffffffffc080200000
Reading symbols from /home/iristseng/data/StarryOS/target/riscv64gc-unknown-none-elf/release/starryos...
```

2. 下一断点组解析成功（此处下一断点组为用户态程序）：

```
al
nc
(i) getStringVariable got: ./test_user
```

3. 用户态程序符号表加载成功：

```
add symbol table from file "/home/iristseng/data/StarryOS/user_apps/test_user" at
.text_addr = 0x10218
Reading symbols from /home/iristseng/data/StarryOS/user_apps/test_user...
```

4. 更详细的调试过程请查看演示视频：

[点击查看：StarryOS 调试演示](#)

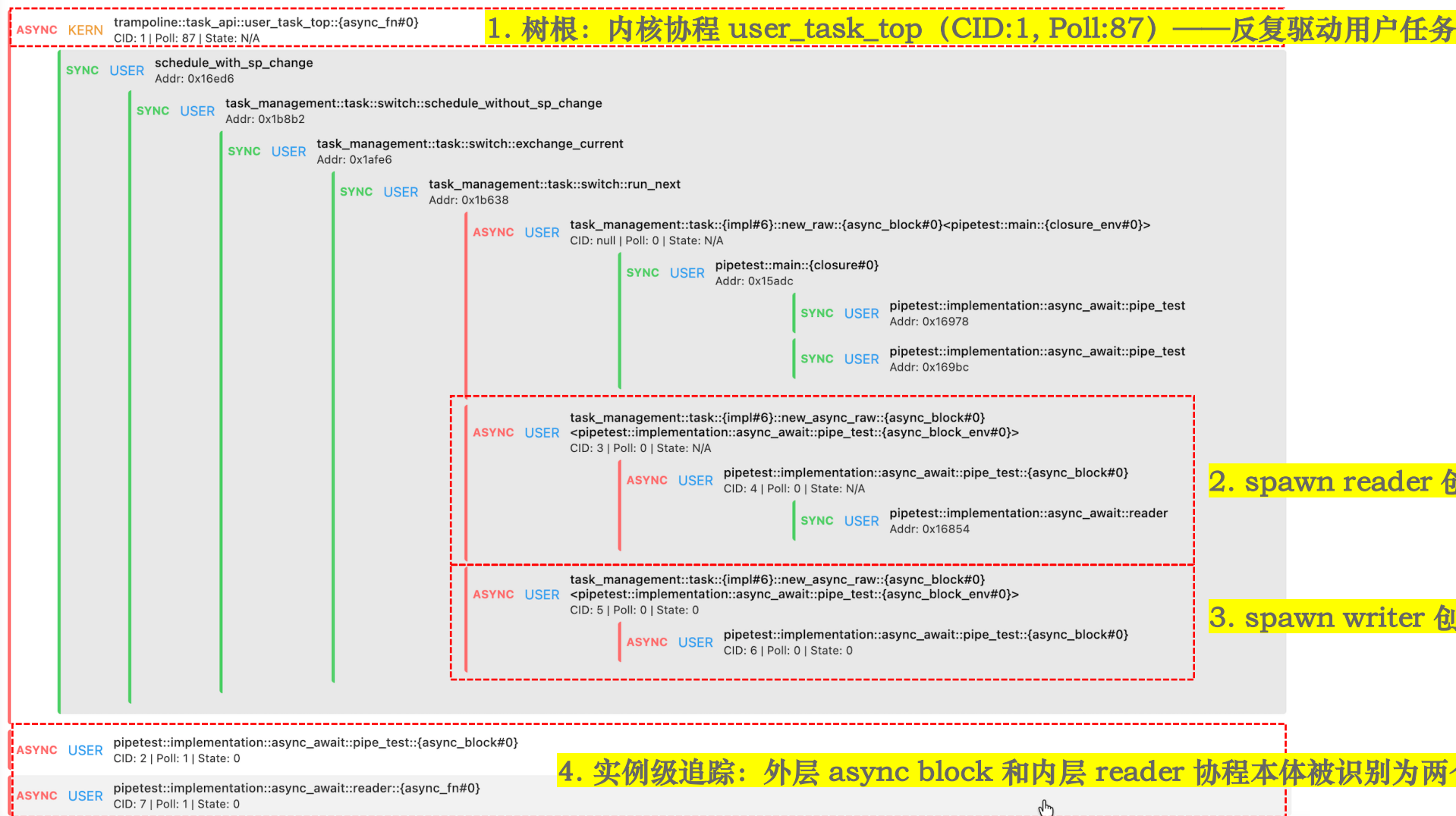
测试验证：异步OS Async-os

验证跨特权级的异步跟踪能力

- **Async-os**: 含异步运行时的 Rust 组件化操作系统，同时涉及特权级切换和异步任务调度。
- **测试目标**: pipetest —— Async-os 自带的管道读写测试程序，覆盖**用户态协程、异步系统调用、特权级切换**。
 - pipe_test 创建两个异步任务：reader（读管道）、writer（写管道）
 - 四个断点打在两个 spawn 点（即 **创建异步任务的入口**）和两个协程入口（即 **运行异步任务的入口**），观察不同执行阶段的树形态

测试验证：异步OS Async-os

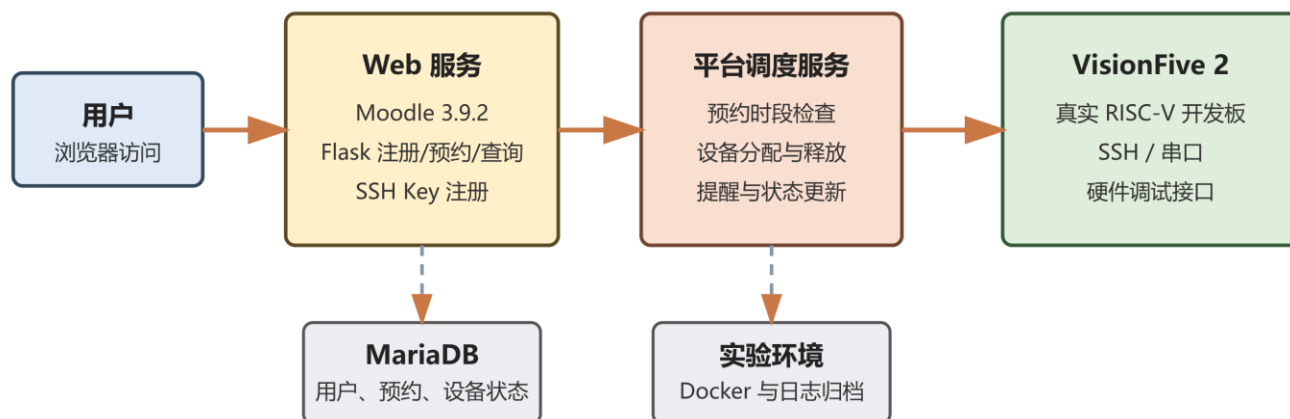
Logical Call Tree



补充：线上调试平台尝试

真实开发板预约调度与远程调试探索

线上硬件交互实验平台结构



当前能力

- 真实 VisionFive 2 的预约、分配与释放
- J-Link、OpenOCD、GDB 与串口链路已建立
- 预约结束后恢复处理器状态并释放设备

正在尝试

- 将 OpenOCD 启停、GDB 连接信息和串口日志纳入平台调度
- 在平台上开展远程操作系统调试
- 将 xv6 调试方法扩展到 StarryOS，并结合 code-debug 验证源码断点、单步与状态切换

无需购买开发板，网页预约即可远程使用 VisionFive 2 完成操作系统源码级调试

参考说明：继承前序工作，核心机制自研

➤ 调试器通信层

GDB/MI2 协议驱动模块移植自开源项目 code-debug: <https://github.com/chenzhiy2001/code-debug>

➤ 断点组管理与跨特权级切换

四状态机驱动的断点组管理机制继承自 code-debug

➤ 异步执行流还原

await edge 与 call edge 双关系恢复方法继承自团队前序工作 ARDB: https://github.com/OSDebugger/code-debug_Asynchronous-trace

➤ 详细参考说明可见仓库决赛文档



北京工商大学
BEIJING TECHNOLOGY AND BUSINESS UNIVERSITY

感谢各位老师 敬请批评指正

团队成员：曾小红 · 王浩铭 · 武雪妍 指导教师：吴竞邦

北京工商大学 · 计算机与人工智能学院 · 2026 年 8 月